Food Delivery Management System

1. Introduction

A Food Delivery Management System is a software application that allows customers to order food from different restaurants through an online platform. It connects customers, restaurants, delivery riders, and administrators in one system.

Customers can browse available restaurants, view food menus, select their favorite meals, place orders, and make payments. Restaurants receive orders and prepare the selected food, while delivery riders transport orders to customers.

The main purpose of this system is to make food ordering faster, easier, and more convenient. It also helps restaurants manage orders efficiently and improves communication between all participants.

2. Identification of Classes and Their Responsibilities

In object-oriented programming, a class is a blueprint used to create objects. The following classes can be used to develop a Food Delivery Management System.

No.	Class Name	Responsibility
1	UserAccount	Stores login credentials and user account information.
2	CustomerProfile	Maintains customer details, contact information, and saved addresses.
3	RestaurantManager	Maintains restaurant information and manages restaurant operations.
4	MenuCatalog	Organizes food categories and menu entries.
5	Meal	Stores details about individual food products, including name, price, and availability.
6	ShoppingBasket	Holds selected meals and calculates the estimated order cost.
7	OrderManager	Creates orders and manages their details and current status.
8	PaymentProcessor	Processes payments and records transaction results.
9	RiderManager	Maintains delivery rider information and assigns deliveries.
10	FeedbackManager	Collects customer ratings, reviews, and suggestions.

These classes divide the system into manageable components. Each class performs a specific responsibility, making the software easier to develop, test, and maintain.

3. Three Major Classes: Attributes and Methods

Attributes represent the data stored by an object, while methods define the operations that the object can perform. The following three classes are essential for the system.

A. CustomerProfile Class

Purpose: This class manages customer information and provides the details required to place and receive food orders.

Attributes:

- customerID — A unique identification number for each customer.
- fullName — The customer's complete name.
- phoneNumber — The customer's contact number.
- emailAddress — The customer's email address.
- deliveryAddress — The location where the order should be delivered.

Methods:

- registerCustomer() — Creates a new customer account.
- updateProfile() — Changes the customer's personal information.
- saveAddress() — Adds or updates a delivery address.
- viewOrderHistory() — Displays the customer's previous orders.
- cancelOrder() — Requests cancellation of an eligible order.

Example: A customer registers an account, saves a home address, and views previous food orders through the system.

B. OrderManager Class

Purpose: This class controls the creation and management of customer orders. It connects selected meals with order details and tracks the progress of each order.

Attributes:

- orderID — A unique number assigned to an order.
- customerID — The identification number of the customer who placed the order.
- selectedMeals — A list of meals included in the order.
- totalAmount — The total cost of the selected items.
- orderStatus — The current condition of the order.
- orderDate — The date and time when the order was created.

Methods:

- createOrder() — Creates an order using the selected meals.
- calculateTotal() — Calculates the total cost of the order.
- confirmOrder() — Confirms an order after the required checks.
- updateOrderStatus() — Updates the order's current status.

- cancelOrder() — Cancels an order when cancellation is permitted.

Example: When a customer confirms a basket containing two burgers and one drink, the system creates an order, calculates its total cost, and records its status.

C. PaymentProcessor Class

Purpose: This class handles payment processing and maintains the result of each payment transaction.

Attributes:

- transactionID — A unique identifier for the payment transaction.
- orderID — The order associated with the payment.
- paymentAmount — The amount to be paid.
- paymentMethod — The selected payment option.
- paymentStatus — Indicates whether the payment is pending, successful, or failed.

Methods:

- initiatePayment() — Starts the payment process.
- validatePayment() — Checks the required payment information.
- confirmPayment() — Records a successful payment confirmation.
- generateReceipt() — Creates a payment receipt.
- refundPayment() — Initiates a refund when permitted.

Example: A customer selects online payment. The system processes the transaction and updates the payment status according to the payment provider's response.

4. Application of Abstraction in the System

Abstraction is an important principle of object-oriented programming. It means hiding unnecessary implementation details and presenting only the essential operations to the user.

In the Food Delivery Management System, customers do not need to understand the internal code used to place an order, calculate its price, or process a payment.

For example, a customer can select the placeOrder() operation without knowing how the system stores order information in a database. Similarly, the customer can use makePayment() without needing to understand the technical communication between the application and the payment service.

Examples of abstraction include:

1. **Order Placement:** The customer selects meals and confirms the order without handling the internal order-processing logic.
2. **Payment Processing:** The user chooses a payment method while the system handles the technical transaction steps.

3. **Delivery Tracking:** The customer views the current delivery status without accessing the rider's internal assignment records.
4. **Menu Browsing:** The customer views available meals and prices without knowing how the restaurant's menu is stored or retrieved.

Benefits of abstraction:

- Reduces complexity for users and developers.
- Makes the system easier to understand.
- Improves software maintenance.
- Separates user operations from internal implementation.
- Helps organize the program into well-defined components.

5. Proposed Software Object: DeliveryScheduler

A proposed software object for the Food Delivery Management System is **DeliveryScheduler**. Its purpose is to coordinate delivery assignments by matching confirmed orders with available delivery riders.

The object helps organize delivery activities and reduces the need for manual assignment.

Attributes:

- scheduleID — A unique identifier for a delivery schedule.
- orderID — The order assigned for delivery.
- riderID — The identification number of the assigned rider.
- pickupLocation — The restaurant location where the food will be collected.
- destinationAddress — The customer's delivery address.
- estimatedDeliveryTime — The expected delivery time.

Methods:

- assignRider() — Assigns an available rider to an order.
- setPickupLocation() — Records the restaurant pickup location.
- calculateEstimatedTime() — Estimates the delivery duration using available route and traffic information.
- rescheduleDelivery() — Updates the delivery schedule when necessary.
- completeDelivery() — Records that the delivery has been completed.

Working Example:

Suppose a customer places an order from a nearby restaurant. After the restaurant confirms the order, the DeliveryScheduler identifies an available rider and assigns the delivery. It stores the pickup location and destination address and provides an estimated delivery time. Once the rider delivers the food, the system updates the delivery record.

This object improves delivery coordination, supports better time management, and helps customers receive more accurate delivery updates.

6. Advantages of the Food Delivery Management System

1. **Convenience:** Customers can order food without visiting restaurants physically.
2. **Time Saving:** Online ordering reduces the time required to select and purchase meals.
3. **Better Organization:** Separate classes manage customers, orders, payments, and deliveries.
4. **Order Monitoring:** Customers can check the progress of their orders.
5. **Improved Communication:** The system connects restaurants, customers, and delivery riders.
6. **Reduced Manual Work:** Automatic calculations and digital order records reduce repetitive tasks.
7. **Customer Satisfaction:** An organized ordering process can improve the overall customer experience.

7. Conclusion

The Food Delivery Management System demonstrates how object-oriented programming can be applied to a practical real-world problem. By dividing the application into classes such as CustomerProfile, OrderManager, PaymentProcessor, and DeliveryScheduler, the system becomes easier to organize and maintain.

Attributes store the information required by each class, while methods define the operations that the system performs. Abstraction simplifies complex processes by providing users with straightforward operations.

In conclusion, a well-designed Food Delivery Management System can improve order management, payment handling, restaurant coordination, and delivery services. It provides a useful example of how software engineering principles can be used to develop efficient and user-friendly applications.